Entry Header

Name:
Aswina Sivaloganathan

Role:
Programmer

Short description:
I implemented a low energy warning pulse effect to visually communicate when the player's energy becomes critically low. The effect creates a pulsing red overlay along the edges of the screen and displays a “LOW ENERGY” warning message.

Small decisions or changes made:
- Chose a pulsing red screen edge effect instead of a full-screen overlay so that gameplay
- Set a threshold value for energy to trigger the warning so it only appears when energy is critically low.

Evidence:
- Code added to main.js to detect low energy levels and trigger the pulse effect.
- Integration with the existing game update and draw cycle.

GenAI Documentation
GenAI tools were used for this task.

Date Used:
March 9th, 2026

Tools, Resources, Inputs Used

- ChatGPT 5.2
- GBDA 302 Week 6 Example Code

Purpose of use:
ChatGPT was used to brainstorm ideas for adding a visual effect tied to the player’s energy mechanic and to generate an example implementation of a pulsing warning overlay. The generated code helped demonstrate how to create a pulsing animation.

Human decision point(s):

I decided which idea to implement from the suggested options and adapted the generated code to match the structure of my project. I also determined where the effect should be placed and how the warning should visually appear to the player.

Integrity and verification note:

All generated code was reviewed and tested manually before being integrated into the project. The implementation was adjusted to ensure compatibility with the existing game architecture and to confirm that the visual effect worked correctly during gameplay.

Limitations or misfires:

The initial suggestions required some adjustment because the structure of my game differed slightly from the example provided

Appendix - Transcript

Add sound or simple physics to your sketch e.g., visual ehects triggered by motion, collisions, or player actions. // main.js

```
// Sketch entry point (VIEW + orchestration layer).
//
// Responsibilities:
// - Load tuning.json and levels.json via LevelLoader
// - Preload assets (images, animations, audio, parallax layers)
// - Create Canvas and configure pixel-perfect rendering
// - Instantiate and wire core systems (Game + input/sound/debug)
// - Draw VIEW elements (background colour, parallax, HUD composite)
// - Own VIEW setup: canvas size, integer scaling, parallax draw, HUD composite
// - Boot the WORLD: load JSON, preload assets, create Game + systems
//
// Non-goals:
// - Does NOT implement gameplay rules (WORLD logic lives in Level/entities)
// - Does NOT manage camera logic inside world update (VIEW modules do)
// - Does NOT contain entity behavior or physics setup beyond global world settings
//
// Architectural notes:
// - main.js owns VIEW setup (canvas sizing, scaling, parallax, background colour).
// - Game owns WORLD orchestration (EventBus, Level lifecycle, system wiring).
// - world.autoStep = false for stable pixel rendering; world.step() happens during world
update.
//
// Important:
// - This file is loaded as a JS module (type="module").
// - In module scope, p5 will NOT automatically find setup/draw.
```

```
//   We MUST attach setup/draw (and input callbacks) to window.
//
// Notes:
// - Browsers block audio autoplay. We unlock audio on the first click/key press.
//
// Dependencies (loaded in index.html before this file):
// - p5.js
// - p5.sound (optional but required for loadSound)
// - p5play

import { LevelLoader } from "./src/LevelLoader.js";
import { Game } from "./src/Game.js";
import { ParallaxBackground } from "./src/ParallaxBackground.js";
import { loadAssets } from "./src/AssetLoader.js";
import { applyIntegerScale, installResizeHandler } from "./src/utils/IntegerScale.js";

import { CameraController } from "./src/CameraController.js";
import { InputManager } from "./src/InputManager.js";
import { SoundManager } from "./src/SoundManager.js";
import { DebugOverlay } from "./src/DebugOverlay.js";

import { WinScreen } from "./src/ui/WinScreen.js";
import { LoseScreen } from "./src/ui/LoseScreen.js";

// ------------------------------------------------------------
// Helpers
// ------------------------------------------------------------

// p5 loadJSON is callback-based. This wrapper lets us use async/await reliably.
function loadJSONAsync(url) {
  return new Promise((resolve, reject) => {
    loadJSON(url, resolve, reject);
  });
}

// Browsers block audio until a user gesture.
// We unlock it once and never think about it again.
let audioUnlocked = false;
function unlockAudioOnce() {
  if (audioUnlocked) return;
```

```
  audioUnlocked = true;
  if (typeof userStartAudio === "function") userStartAudio();
}

// Prevent the browser from stealing keys (space/arrows) for scrolling.
function preventKeysThatScroll(evt) {
  const k = (evt?.key ?? "").toLowerCase();
  const scrollKeys = [" ", "arrowup", "arrowdown", "arrowleft", "arrowright"];
  if (scrollKeys.includes(k)) {
    evt.preventDefault?.();
    return false;
  }
  return true;
}

// ------------------------------------------------------------
// State (WORLD + VIEW glue)
// ------------------------------------------------------------

let game; // WORLD orchestrator (updates + draws world)
let parallax; // VIEW background parallax
let hudGfx; // VIEW overlay buffer (screen-space)

let tuningDoc; // Data: tuning.json
let levelPkg; // Data package from LevelLoader (level + view + world + tiles)
let assets; // Preloaded assets bundle

let cameraController; // VIEW: follow + clamp camera to world bounds
let inputManager; // SYSTEM: keyboard snapshot
let soundManager; // SYSTEM: audio registry
let debugOverlay; // VIEW/SYSTEM: debug UI

let winScreen;
let loseScreen;
let parallaxLayers = []; // Preloaded parallax layer defs [{ img, factor }, ...]

// Make URLs absolute so they can't accidentally resolve relative to /src/...
const LEVELS_URL = new URL("./data/levels.json", window.location.href).href;
const TUNING_URL = new URL("./data/tuning.json", window.location.href).href;
```

```
// This must match a level id in levels.json
const START_LEVEL_ID = "ex5_level1";

// Boot flags
let bootStarted = false;
let bootDone = false;

// ----------------------------------------------------------
// Boot pipeline (async) — runs from setup()
// ----------------------------------------------------------

async function boot() {
  console.log("BOOT: start");

  // --- Data ---
  tuningDoc = await loadJSONAsync(TUNING_URL);

  const loader = new LevelLoader(tuningDoc);
  levelPkg = await loader.load(LEVELS_URL, START_LEVEL_ID);

  // --- Assets (images/animations/etc.) ---
  assets = await loadAssets(levelPkg, tuningDoc);

  // --- Audio registry ---
  // (AudioContext may still be locked until the user clicks/presses a key.)
  soundManager = new SoundManager();

  // --- Parallax layer defs (VIEW) ---
  const defs = levelPkg.level?.view?.parallax ?? [];
  parallaxLayers = defs
    .map((d) => ({
      img: loadImage(d.img),
      factor: Number(d.speed ?? 0),
    }))
    .filter((l) => l.img);

  // Now that all data is ready, build the WORLD + VIEW runtime.
  initRuntime();

  bootDone = true;
```

```
  console.log("BOOT: done");
}

// ---------------------------------------------------------
// Runtime init (sync) — called after boot() finishes
// ---------------------------------------------------------

function initRuntime() {
  const { viewW, viewH } = levelPkg.view;

  // Resize the tiny placeholder canvas created in setup().
  resizeCanvas(viewW, viewH);

  // Pixel art: never smooth, never retina-scale the main canvas.
  pixelDensity(1);
  noSmooth();
  drawingContext.imageSmoothingEnabled = false;

  // Keep timing stable (p5play anims feel best when p5 is targeting 60).
  frameRate(60);

  // Pixel-perfect scaling to fill the browser window by integer multiples.
  applyIntegerScale(viewW, viewH);
  installResizeHandler(viewW, viewH);

  // Sprite rendering
  allSprites.pixelPerfect = true;

  // Physics: manual step for stable pixel rendering
  world.autoStep = false;

  // HUD buffer (screen-space)
  hudGfx = createGraphics(viewW, viewH);
  hudGfx.noSmooth();
  hudGfx.pixelDensity(1);

  // Systems
  inputManager = new InputManager();
  debugOverlay = new DebugOverlay();
```

```
  // WORLD
  game = new Game(levelPkg, assets, {
    hudGfx,
    inputManager,
    soundManager,
    debugOverlay,
  });
  game.build();

  // UI overlays
  winScreen = new WinScreen(levelPkg, assets);
  loseScreen = new LoseScreen(levelPkg, assets);

  // VIEW: camera follow + clamp
  cameraController = new CameraController(levelPkg);
  cameraController.setTarget(game.level.playerCtrl.sprite);
  cameraController.reset();

  // IMPORTANT: subscribe ONCE (not in draw)
  game.events.on("level:restarted", () => {
    cameraController?.reset();
  });

  // VIEW: parallax background renderer
  parallax = new ParallaxBackground(parallaxLayers);

  loop();
}

// ---------------------------------------------------------
// p5 lifecycle (module-safe)
// ---------------------------------------------------------

function setup() {
  // Create a tiny placeholder canvas immediately so p5 is happy,
  // then pause the loop until our async boot finishes.
  new Canvas(10, 10, "pixelated");
  pixelDensity(1);
  noLoop();
```

```
  if (bootStarted) return;
  bootStarted = true;

  boot().catch((err) => {
    console.error("BOOT FAILED:", err);
    // loop stays stopped so the sketch doesn't spam errors
  });
}

function draw() {
  if (!bootDone || !levelPkg || !game) return;

  const viewW = levelPkg.view.viewW;
  const viewH = levelPkg.view.viewH;

  // Background colour is per-level in levels.json: level.view.background
  const bg = levelPkg.level?.view?.background ?? [69, 61, 79];
  background(bg[0], bg[1], bg[2]);

  // Parallax uses camera.x from previous frame (fine with manual stepping)
  parallax?.draw({
    cameraX: camera.x || 0,
    viewW,
    viewH,
  });

  // WORLD update (includes physics step)
  game.update();

  // VIEW: camera follow + clamp (after update so player position is current)
  cameraController?.update({
    viewW,
    viewH,
    levelW: game.level.bounds.levelW,
    levelH: game.level.bounds.levelH,
  });
  cameraController?.applyToP5Camera();

  // WORLD draw + HUD composite
  game.draw({
```

```
    drawHudFn: () => {
      // camera.off()/on() MUST be paired even if something throws.
      camera.off();
      try {
        drawingContext.imageSmoothingEnabled = false;
        imageMode(CORNER);
        image(hudGfx, 0, 0);
      } finally {
        camera.on();
        noTint(); // prevent tint leaking into world next frame
      }
    },
  });

  // ----------------------------
  // VIEW flow gates
  // ----------------------------
  const won = game?.won === true || game?.level?.won === true;
  const dead = game?.lost === true || game?.level?.player?.dead === true;

  // Prefer Game mirror
  const elapsedMs = Number(game?.elapsedMs ?? game?.level?.elapsedMs ?? 0);

  // These overlay draw calls already guard camera.off/on internally,
  // but we keep them outside of any camera.off scope here.
  if (won) winScreen?.draw({ elapsedMs, game });
  if (dead) loseScreen?.draw({ elapsedMs, game });
}

// ----------------------------------------------------------
// Optional input callbacks (audio unlock feels invisible)
// ----------------------------------------------------------

function mousePressed() {
  unlockAudioOnce();
}

function keyPressed(evt) {
  unlockAudioOnce();
  return preventKeysThatScroll(evt);
```

```
}

// Extra safety: prevent scrolling even if p5 doesn't route a key event you expect.
window.addEventListener(
  "keydown",
  (e) => {
    const k = (e.key ?? "").toLowerCase();
    if ([" ", "arrowup", "arrowdown", "arrowleft", "arrowright"].includes(k)) {
      e.preventDefault();
    }
  },
  { passive: false },
);

// ------------------------------------------------------------
// IMPORTANT: expose p5 entrypoints in module scope
// ------------------------------------------------------------

window.setup = setup;
window.draw = draw;
window.mousePressed = mousePressed;
window.keyPressed = keyPressed;  help me brainstorm ideas for this sidequest
```

For this side quest, you basically just need one small interactive effect triggered by something in your game (movement, collision, key press, etc.). Since your game already has a player, movement, energy, keys, and hearts, the easiest way is to add feedback effects. These make the game feel more alive without rewriting your whole system.

Here are some ideas that will work well with your current game architecture 👇

---

1. Footstep Sound When Player Moves (Very Easy) 🔊

Trigger: when the player presses movement keys.

Idea

Every time the player walks left or right, play a quiet footstep sound.

Why it works

- Simple to add
- Uses your existing SoundManager
- Makes movement feel responsive

Logic idea

```
if (player is moving left or right) {
  play footstep sound
}
```

You can even make it play every 0.3 seconds so it doesn't spam.

Even better

Pitch the sound slightly different each time.

Example effect:

- left step
- right step
- slight variation

---

2. Screen Shake When Energy Drops ⚡

Trigger: when the player’s energy suddenly drops.

Idea

Add a tiny camera shake to simulate emotional impact.

Example effect

When energy decreases:

- screen shakes for ~10 frames
- slight blur/dark pulse

Visual concept

player energy drops
→ screen shakes slightly
→ vignette darkens briefly

```
camera.x += random(-2, 2)
camera.y += random(-2, 2)
```

Simple implementation concept

---

3. Heart Pickup Sparkle Effect ✨

Trigger: when player collects a heart.

Idea

Spawn small glowing particles around the heart.

Visual effect

♥ collected
→ small particles burst outward
→ fade after 0.5 seconds

- reinforces positive feedback
- super common game feel effect
- easy to implement with circles

Example pseudo:

```
for i in 10 particles:

  spawn particle at heart position
```

Particles:

- move outward
- shrink
- fade

---

4. Heavy Movement When Energy Low 🐢

Trigger: energy below a threshold.

Idea

When energy is low:

- player moves slower
- sprite tilts slightly
- footsteps sound heavier

Example effect

```
if energy < 30:
  playerSpeed = normalSpeed * 0.6
```

Optional visual:

```
sprite.rotation = sin(frameCount * 0.1) * 2
```

This reinforces your mental health mechanic.

---

5. Key Pickup Sound + Glow 🔑

Trigger: when key collected.

Effect

- play a ding sound
- key flashes yellow
- small glow expands

Example effect:

```
key collected
```

→ yellow ring expands

→ fade out

Visual code concept:

ellipse(key.x, key.y, radius++)

---

6. Player Breathing Animation When Idle 😮‍💨

Trigger: when the player stops moving.

Effect

The character subtly scales up and down.

Example:

scale = 1 + sin(frameCount * 0.05) * 0.03

Makes the character feel alive.

---

7. Low Energy Warning Pulse ❤️‍🩹

Trigger: energy very low.

Effect

- screen edges pulse red
- heartbeat sound

Example effect:

```
if energy < 20:

  vignetteAlpha = sin(frameCount*0.2) * 100
```

⭐ Easiest Side Quest (Recommended)

If you want something fast and clean, do:

Heart pickup particle burst + sound

It shows:

- collision detection
- visual effect
- sound
- player feedback

Your prof will immediately see you added physics/visual interaction.

Example Combo (Best submission)

When the player collects a heart:

1. play "heal.wav"

2. spawn 12 particles

3. particles move outward

4. particles fade away

Boom. Side quest done.

✅ If you want, I can also give you:

- a copy-paste particle system (20 lines)
- a sound trigger using your SoundManager
- the cleanest way to hook it into Game.js or Level.js

Just send me the heart collision code or player controller file and I'll wire it in perfectly.

I like the Low Energy Warning Pulse! How can I implement this

Yes — this is a really good one for your game because it matches your mental health / energy mechanic and it's also pretty simple to add.

You can implement the Low Energy Warning Pulse as a screen-space overlay that appears when the player's energy gets low.

What it should do

When energy is below a threshold:

- the screen gets a faint red tint
- it pulses in and out over time
- optional: make it stronger when energy is even lower

Because this is a HUD / overlay effect, the best place is usually in your HUD drawing layer, not inside world/entity code.

Basic idea

You already have this in `main.js`:

1. detect the player's current energy
2. if it is low, draw a pulsing red overlay onto `hudGfx`
3. then your existing HUD render will display it

Simple version

1. Choose a threshold

For example:

```
const LOW_ENERGY_THRESHOLD = 30;
```

If your energy is out of 100, 30 is a good warning point.

---

2. Make a helper function

You can add this function in `main.js` above `draw()`:

```
function drawLowEnergyPulse(hudGfx, energy, maxEnergy, viewW, viewH) {

 const threshold = 30;

 if (energy > threshold) return;



 const t = frameCount * 0.12;

 const pulse = (sin(t) + 1) * 0.5;



 const severity = constrain(1 - energy / threshold, 0, 1);

 const alpha = 20 + pulse * 50 * severity;



 hudGfx.push();

 hudGfx.noStroke();

 hudGfx.fill(255, 0, 0, alpha);

 hudGfx.rect(0, 0, viewW, viewH);

 hudGfx.pop();
```

```
}
```

This gives you:

- no effect when energy is normal
- stronger effect when energy gets lower
- a smooth pulsing animation

---

3. Call it before the HUD gets drawn

Inside `draw()`, before `game.draw(...)`, clear `hudGfx` and draw the pulse.

You can do something like this:

```
function draw() {

 if (!bootDone || !levelPkg || !game) return;



 const viewW = levelPkg.view.viewW;

 const viewH = levelPkg.view.viewH;



 const bg = levelPkg.level?.view?.background ?? [69, 61, 79];

 background(bg[0], bg[1], bg[2]);



 parallax?.draw({

   cameraX: camera.x || 0,

   viewW,
```

```
  viewH,
});


game.update();


cameraController?.update({
  viewW,
  viewH,
  levelW: game.level.bounds.levelW,
  levelH: game.level.bounds.levelH,
});
cameraController?.applyToP5Camera();


hudGfx.clear();


const energy = game?.level?.player?.energy ?? 100;
const maxEnergy = game?.level?.player?.maxEnergy ?? 100;


drawLowEnergyPulse(hudGfx, energy, maxEnergy, viewW, viewH);
```

```
game.draw({
  drawHudFn: () => {
    camera.off();
    try {
      drawingContext.imageSmoothingEnabled = false;
      imageMode(CORNER);
      image(hudGfx, 0, 0);
    } finally {
      camera.on();
      noTint();
    }
  },
});

const won = game?.won === true || game?.level?.won === true;
const dead = game?.lost === true || game?.level?.player?.dead === true;
const elapsedMs = Number(game?.elapsedMs ?? game?.level?.elapsedMs ?? 0);
```

```
 if (won) winScreen?.draw({ elapsedMs, game });

 if (dead) loseScreen?.draw({ elapsedMs, game });

}
```

Better version: vignette pulse instead of full red screen

A full-screen red rectangle works, but a vignette-style warning looks nicer and more dramatic.

Instead of tinting the whole screen evenly, draw red translucent bars around the edges.

Replace the helper with this:

```
function drawLowEnergyPulse(hudGfx, energy, maxEnergy, viewW, viewH) {

 const threshold = 30;

 if (energy > threshold) return;



 const t = frameCount * 0.12;

 const pulse = (sin(t) + 1) * 0.5;

 const severity = constrain(1 - energy / threshold, 0, 1);



 const alpha = 15 + pulse * 60 * severity;

 const edge = 40 + 20 * severity;
```

```
 hudGfx.push();
 hudGfx.noStroke();
 hudGfx.fill(180, 0, 0, alpha);

 hudGfx.rect(0, 0, viewW, edge);
 hudGfx.rect(0, viewH - edge, viewW, edge);
 hudGfx.rect(0, 0, edge, viewH);
 hudGfx.rect(viewW - edge, 0, edge, viewH);

 hudGfx.pop();
}
```

This gives a much better “low health / low energy” feel.

Even nicer: layered vignette

If you want it to look more polished, use several rectangles with fading alpha:

```
function drawLowEnergyPulse(hudGfx, energy, maxEnergy, viewW, viewH) {
 const threshold = 30;
 if (energy > threshold) return;
```

```
const t = frameCount * 0.12;
const pulse = (sin(t) + 1) * 0.5;
const severity = constrain(1 - energy / threshold, 0, 1);


hudGfx.push();
hudGfx.noStroke();


for (let i = 0; i < 4; i++) {
  const edge = 18 + i * 12;
  const alpha = (8 + pulse * 18) * severity * (1 - i * 0.18);


  hudGfx.fill(200, 0, 0, alpha);
  hudGfx.rect(0, 0, viewW, edge);
  hudGfx.rect(0, viewH - edge, viewW, edge);
  hudGfx.rect(0, 0, edge, viewH);
  hudGfx.rect(viewW - edge, 0, edge, viewH);
}
```

```
 hudGfx.pop();

}
```

That one looks more like an actual warning pulse.

Where to get the energy value

You'll need to confirm where your energy is stored.

Common possibilities:

```
game.level.player.energy

game.level.playerCtrl.energy

game.level.playerStats.energy
```

From your earlier code, I'd first try:

```
const energy = game?.level?.player?.energy ?? 100;

const maxEnergy = game?.level?.player?.maxEnergy ?? 100;
```

If that doesn't work, check wherever your energy bar logic currently reads from.

If you want the pulse only when energy is critically low

Use a lower threshold:

```
const threshold = 20;
```

That makes it more intense and less frequent.

Optional extra: add a tiny shake when pulse is strongest

If you want one more effect without overdoing it, when energy is super low you can slightly jitter the HUD:

```
const shake = energy < 10 ? random(-1, 1) : 0;

image(hudGfx, shake, shake);
```

But honestly, for the side quest, the pulse alone is enough.

Best simple version to submit

I'd recommend this:

- red edge pulse
- starts below 30 energy
- gets stronger as energy drops

That's easy to explain in your documentation too:

I added a low-energy warning pulse that visually communicates when the player's emotional capacity is running low. As energy decreases, a red pulsing screen-edge overlay intensifies, reinforcing urgency and making the mechanic more visible to the player.

What to add to your `main.js`

You can copy this helper directly:

```
function drawLowEnergyPulse(hudGfx, energy, maxEnergy, viewW, viewH) {

 const threshold = 30;
```

```
if (energy > threshold) return;



const t = frameCount * 0.12;

const pulse = (sin(t) + 1) * 0.5;

const severity = constrain(1 - energy / threshold, 0, 1);



hudGfx.push();

hudGfx.noStroke();



for (let i = 0; i < 4; i++) {

  const edge = 18 + i * 12;

  const alpha = (8 + pulse * 18) * severity * (1 - i * 0.18);



  hudGfx.fill(200,
```